AIORI Hackathon 3

October 14, 2026

A3-PS021-TC516

Threadline: Experimental Dialog Correlation and Recovery Across Agent Adapters (PS-021)

Yugraj Mangate

github.com/yugrajmangate-dev

Backend, API, SQLite persistence, coordination

Neha Niphade

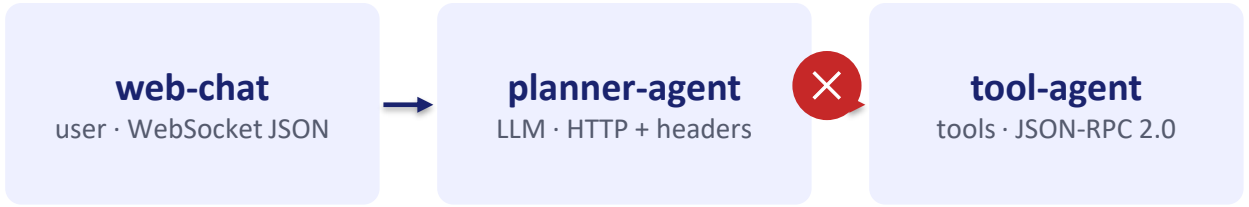
github.com/Neha-Niphade

Adapters, correlation, recovery, tests, dashboard

Team Nexora · Code: github.com/yugrajmangate-dev/A3-PS021-TC516 · Live demo: threadline-brown.vercel.app

Problem Description

One AI conversation crosses agents that speak different protocols, and breaks when delivery fails.



Hand-off to the tool agent times out. What now?

What goes wrong without correlation

- Context is lost: the resumed agent does not know what was already said
- A retried step produces a second assistant reply
- Two agents answer the same step (split-brain)
- Gaps are silently skipped; nobody can explain why

Failure modes PS-021 asks us to handle

Adapter timeout
interrupted hand-off

Duplicates
redelivery & retries

Out of order
seq 3 before seq 2

Delayed
arrives after recovery

Lost
a sequence never arrives

Split-brain
two claims, one slot

Background

Multi-agent systems are distributed systems: no shared clock, at-least-once delivery, retries everywhere.

1 Heterogeneous adapters

Chat widgets (WebSocket), LLM agents (HTTP) and tool servers (JSON-RPC 2.0, as in MCP) each carry correlation in a different place [5][6][10].

2 At-least-once delivery

Retries and redelivery make duplicates normal. Exactly-once processing has to be built from idempotency keys and sequence numbers [1][2][9].

3 No trustworthy global time

Wall clocks differ between services and retries re-stamp messages, so order must come from sequence and causation, not timestamps [3].

Key insight The adapters must not own the conversation. An append-only event log does, and recovery is a deterministic replay of that log [7][8].

Literature & RFC Review

The standards and studies we build on, and what we take from each.

Source	What it establishes	How Threadline applies it
RFC 9110 HTTP Semantics [1]	Idempotent methods may be retried safely	Every dialog step is safe to retry: a repeat is detected, not reprocessed
Idempotency-Key header (IETF draft) [2]	Client key deduplicates retried POSTs	Key derived from the dialog slot (conversation, seq, role, cause, kind), verified on replay
Lamport, logical clocks [3]	Order by happened-before, not wall clocks	Order by sequence + causation_id; timestamps never decide order
W3C Trace Context [4]	Correlation ids propagated across services	Five correlation fields survive all three wire protocols
Event Sourcing [7] · Sagas [8]	State = replay of events; multi-step recovery	Append-only SQLite log; pure replay(); resume from a safe checkpoint
Kafka KIP-98 [9]	Producer sequence numbers enable broker dedup	Sequence slots + idempotency key drop redeliveries and retries

Proposed Solution

Correlate every message, log every delivery, recover by deterministic replay.

Canonical envelope (every adapter)

- `conversation_id` stable dialog id
- `message_id` unique per message
- `sequence` position in the dialog
- `causation_id` message it answers
- `idempotency_key` hash(conv, seq, role, cause, kind)

No timestamps, no adapter names in identity

1. **Correlate**

   Three protocols (WebSocket JSON, HTTP headers, JSON-RPC 2.0) decode into one envelope.

2. **Log**

   Gateway appends every delivery to SQLite; triggers block UPDATE/DELETE on the event log.

3. **Recover**

   Pure replay(): validate, dedupe, resolve conflicts, order, verify gaps. Verdict + rule + reason per event.

4. **Resume**

   Continue an open hand-off from the recovered checkpoint. A gap blocks resume: never invent context.

Optimization Proposed

From the first prototype to Threadline: what changed and why.

Aspect	Before (first prototype)	After (Threadline)
Idempotency key	conversation:sequence:adapter, so the same step via another adapter got a new key	**hash(conversation, seq, role, cause, kind): retries across adapters collapse**
Interrupted hand-off	Reported 'recovered'; the open hand-off was silently ignored	**Detected, linked to the timeout signal, resumable by the target adapter**
Missing messages	Never surfaced	**Gaps listed, status incomplete, resume blocked**
Ordering	Ties broken by timestamps	**Sequence order; log offset breaks ties**
Causation	Pointed at the delivery event	**Points at the message it answers**
Persistence	In-memory dict, lost on restart	**Append-only SQLite log with triggers**
Fault coverage	No duplicates, reordering or gaps produced	**6 composable faults, 8 deterministic scenarios**

Optimization Proposed II: Measured Results

python -m threadline.bench: every scenario through run → recover → deliver → resume → recover twice.

Scenario	Faults injected	First recovery	Final status	Dialog	Same hash
Happy path	none	consistent	consistent	complete	yes
Interrupted hand-off	timeout at seq 7	recovered	recovered (+2 resumed)	complete	yes
Duplicate storm	redelivery + retry	recovered (2 removed)	recovered	complete	yes
Out of order	3↔2, 9↔8	recovered (2 reordered)	recovered	complete	yes
Delayed	seq 6 in transit	incomplete (gap 6)	recovered	complete	yes
Missing	seq 5 lost	incomplete (gap 5)	incomplete	ended_with_gaps	yes
Split-brain	two claims on seq 4	recovered_with_conflicts	recovered_with_conflicts	complete	yes
Chaos	5 faults combined	incomplete (gap 6)	recovered_with_conflicts	complete	yes

8 / 8
scenarios idempotent

0
messages invented

50
automated tests

< 1 ms
replay per dialog

Solution Architecture & Design

Components and dependencies (standard-library Python, zero third-party packages)

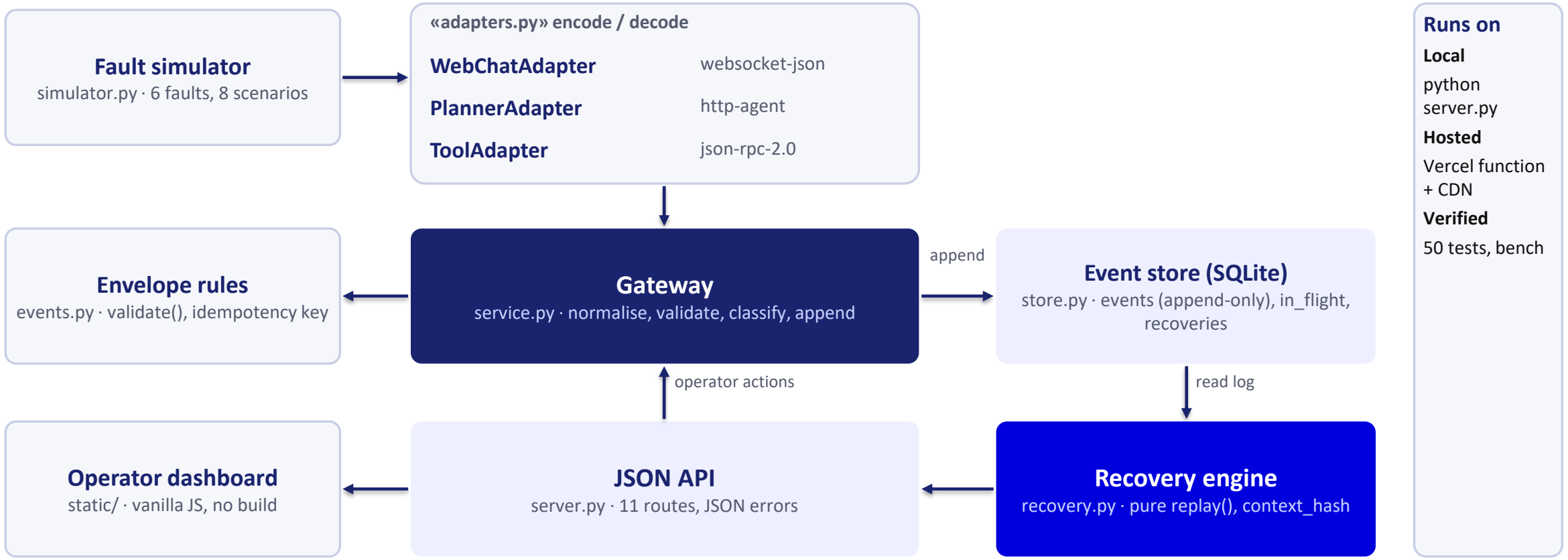


Figure: System Architecture UML Diagram

Data Flow Diagram

The ingest path writes evidence; the recovery path only reads it

INGEST (every delivery)

Native payload
WS frame / HTTP / JSON-RPC

→ **Decode**
adapter codec

→ **Normalise + validate**
derive & check key

→ **Classify live**
accepted / duplicate / conflict

→ **Append**
never filtered

↓

Append-only log

RECOVERY (operator: Replay & recover)

Validate → **Deduplicate** → **Resolve conflicts** → **Order by seq** → **Verify gaps** → **Resume point**

OUTPUTS

Recovered context	Decisions	context_hash	Resume / block
ordered, deduplicated dialog	verdict + rule + reason per event	same log → same hash	next seq + adapter, or the gap

Figure: Data Flow Diagram (DFD)

Deployment Timeline

Milestones and method: design the rules first, then prove each one with deterministic tests.



Design

Dialog lifecycle, envelope and recovery rules agreed as one team

Correlate

Envelope, idempotency key, 3 protocol adapters

Persist

Gateway + append-only SQLite log (MVP, Oct 11)

Recover

replay(), fault simulator, 8 scenarios, 50 tests

Deploy

Dashboard, JSON API, live on Vercel (Oct 14)

Submit

Orientation video, slides, AIORI portal

Next Real connectors (WebSocket, LLM HTTP, JSON-RPC / MCP) · outbox redelivery to heal gaps · snapshots for long logs · seeded random fault campaigns · signed envelopes · PostgreSQL · distributed sequencer

Bibliography I

[1] R. Fielding, M. Nottingham, J. Reschke. *HTTP Semantics.* RFC 9110, IETF, June 2022.

[2] J. Jena, S. Dalal. *The Idempotency-Key HTTP Header Field.* Internet-Draft draft-ietf-httpapi-idempotency-key-header, IETF (work in progress).

[3] L. Lamport. *Time, Clocks, and the Ordering of Events in a Distributed System.* Communications of the ACM 21.7 (1978), pp. 558–565.

[4] W3C. *Trace Context.* W3C Recommendation, 2021.

[5] JSON-RPC Working Group. *JSON-RPC 2.0 Specification.* 2010 (rev. 2013).

Bibliography II

[6] I. Fette, A. Melnikov. *The WebSocket Protocol.* RFC 6455, IETF, December 2011.

[7] M. Fowler. *Event Sourcing.* martinfowler.com, 2005.

[8] H. Garcia-Molina, K. Salem. *Sagas.* Proc. ACM SIGMOD (1987), pp. 249–259.

[9] Apache Kafka. *KIP-98: Exactly Once Delivery and Transactional Messaging.* 2017.

[10] Model Context Protocol. *Specification (JSON-RPC 2.0 based).* modelcontextprotocol.io.



Thank You for Your Attention!

Questions welcome

Code
github.com/yugrajmangate-dev/A3-PS021-TC516

Live demo
threadline-brown.vercel.app

Orientation video
youtu.be/4IXSzTUrEOA

Run locally
python server.py → localhost:8080